



I'm working on a lab assignment: a Pygame "Real-Time Reaction Time Tester".

Here is the README with the tasks:

Task 1: Refine Input Timing Detection

Reaction times are measured from when the round started, not from when the screen actually turned green - so every recorded time is inflated by however long the wait phase lasted, and clicking during the grey "wait" screen gets timed and recorded exactly like a real reaction instead of being flagged as a false start. Investigate and enhance input handling so only genuine reactions after "go" are recorded, measured from the moment "go" happened.

Task 2: Implement Game Over Condition

Add a screen that displays the final results (every reaction time and the average) once all rounds are complete, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

After the results screen, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard wait-time range/round count), or exit.

Task 4: Add Sound Feedback

Add basic sound effects for the "go" cue, a false start, and the session ending.

Here is the current code:— main.py —

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 400
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
pygame.display.set_caption("Reaction Time Tester - Pygame  
Version")
```

```
# Clock
```

```
clock = pygame.time.Clock()
```

```
FPS = 60
```

```
# Game loop
```

```
engine = GameEngine(WIDTH, HEIGHT)
```

```
def main():
```

```
    running = True
```

```
    while running:
```

```
        for event in pygame.event.get():
```

```
            if event.type == pygame.QUIT:
```

```
                running = False
```

```
            engine.handle_event(event)
```

```
        engine.handle_input()
```

```
        engine.update()
```

```
        engine.render(SCREEN)
```

```
        pygame.display.flip()
```

```
        clock.tick(FPS)
```

```
pygame.quit()
```

```
if __name__ == "__main__":
```

```
    main()
```

```
— game/game_engine.py —
```

```
import pygame
```

```
from .round import Round
```

```
# Game Engine
```

```
WHITE = (255, 255, 255)
```

```
BLACK = (0, 0, 0)
```

```
GRAY = (90, 90, 90)
```

GREEN = (40, 180, 90)

BLUE = (50, 90, 170)

class GameEngine:

def __init__(self, width, height, rounds_total=5,
min_wait_ms=1000, max_wait_ms=3000):

self.width = width

self.height = height

self.rounds_total = rounds_total

self.min_wait_ms = min_wait_ms

self.max_wait_ms = max_wait_ms

self.round = Round(self.min_wait_ms, self.max_wait_ms)

self.reaction_times = []

self.result_shown_at = None

self.result_pause_ms = 800 # brief pause on the result screen
between rounds

self.font = pygame.font.SysFont("Arial", 30)

self.big_font = pygame.font.SysFont("Arial", 46)

self.game_over = False

def handle_event(self, event):

if self.game_over:

return

is_click = event.type == pygame.MOUSEBUTTONDOWN

is_space = event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE

if (is_click or is_space) and self.round.state != "result":

reaction_ms = self.round.register_input()

self.reaction_times.append(reaction_ms)

self.result_shown_at = pygame.time.get_ticks()

def handle_input(self):

Reserved for continuously-held-key input; every action here

is a discrete click/keypress, handled in handle_event.

pass

def update(self):

```

    if self.game_over:
        return

    self.round.update()

    if self.round.state == "result":
        now = pygame.time.get_ticks()
        if now - self.result_shown_at >= self.result_pause_ms:
            self._start_next_round()

    def _start_next_round(self):
        if len(self.reaction_times) >= self.ounds_total:
            self.game_over = True
            return
        self.round = Round(self.min_wait_ms, self.max_wait_ms)

    def average_reaction_ms(self):
        if not self.reaction_times:
            return 0
        return round(sum(self.reaction_times) /
len(self.reaction_times))

    def render(self, screen):
        if self.round.state == "waiting":
            bg = GRAY
            message = "Wait for green..."
        elif self.round.state == "go":
            bg = GREEN
            message = "Click now!"
        else:
            bg = BLUE
            message = f"{self.round.reaction_ms} ms"

        screen.fill(bg)

        text_surf = self.big_font.render(message, True, WHITE)
        text_rect = text_surf.get_rect(center=(self.width // 2, self.height
// 2))
        screen.blit(text_surf, text_rect)

        round_num = min(len(self.reaction_times) + 1,

```

```

self.rounds_total)
    round_text = self.font.render(f"Round
{round_num}/{self.rounds_total}", True, WHITE)
    screen.blit(round_text, (10, 10))

    avg_text = self.font.render(f"Avg: {self.average_reaction_ms()}
ms", True, WHITE)
    screen.blit(avg_text, (self.width - 190, 10))

    if self.game_over and not getattr(self, "_game_over_logged",
False):
        # NOTE: no proper results screen yet - see Task 2 in the
README.
        print("Session complete! Reaction times (ms):",
self.reaction_times)
        print("Average:", self.average_reaction_ms(), "ms")
        self._game_over_logged = True

```

— game/round.py —

```

import random
import pygame

class Round:
    def __init__(self, min_wait_ms=1000, max_wait_ms=3000):
        self.wait_delay_ms = random.randint(min_wait_ms,
max_wait_ms)
        self.state = "waiting" # "waiting" -> "go" -> "result"
        self.start_time = pygame.time.get_ticks()
        self.go_time = None
        self.reaction_ms = None

    def update(self):
        if self.state == "waiting":
            now = pygame.time.get_ticks()
            if now - self.start_time >= self.wait_delay_ms:
                self.state = "go"
                self.go_time = now

    def register_input(self):

```

```
# NOTE: this always measures elapsed time since the round
# STARTED (self.start_time), not since the screen actually
# turned green (self.go_time) - and it never checks self.state
# first. Two consequences: (1) a click during the grey
# "waiting" phase is timed and recorded exactly like a real
# reaction instead of being flagged as a false start, and (2)
# even a genuine reaction after "go" is inflated by however
# long the wait phase lasted, since the clock never resets
# when the screen turns green. See Task 1 in the README.
now = pygame.time.get_ticks()
self.reaction_ms = now - self.start_time
self.state = "result"
return self.reaction_ms
```

Don't change anything yet. First explain how the game currently works, and identify the bug described in Task 1 (timing measured from round start instead of the green "go" moment, and clicks during the grey wait screen being recorded as valid reactions). Point to the exact lines responsible.



How the game currently works

Structure

There are three pieces:

- **main.py** runs the Pygame loop at 60 FPS. Each frame it forwards every event to `engine.handle_event`, then calls `handle_input()` (a no-op), `update()`, and `render()`.
- **GameEngine** owns the session: the list of `reaction_times`, the number of rounds, the current Round, and the `game_over` flag.
- **Round** is a small state machine for a single attempt: `"waiting"` → `"go"` → `"result"`.

Lifecycle of one round

1. **Creation.** `Round.__init__` picks a random wait delay (1000–3000 ms) and stores `self.start_time = pygame.time.get_ticks()` . The state is "waiting" and the screen is grey.
2. **Waiting → go.** Each frame, `Round.update()` checks whether `now - start_time >= wait_delay_ms` . When it has, it sets `state = "go"` and records `go_time = now` . The screen turns green.
3. **Input.** On a click or spacebar press, `GameEngine.handle_event` calls `self.round.register_input()` , appends the returned value to `reaction_times` , and stamps `result_shown_at` .
4. **Result.** `register_input` sets `state = "result"` and the blue screen shows the time. After 800 ms, `GameEngine.update` calls `_start_next_round()` .
5. **Next round or end.** If `len(reaction_times) >= rounds_total` , `game_over = True` . Otherwise a fresh `Round` is created. At game over, `render` prints the results to the console once and the screen stays frozen on the last round's blue result.

The Task 1 bug

There are two related defects, and both come from the same missing logic in `Round.register_input()` (`game/round.py`).

Defect 1: wrong time base

python

```
now = pygame.time.get_ticks()
self.reaction_ms = now - self.start_time
```

`start_time` is when the round was created, which is when the grey screen first appeared. The correct reference is `self.go_time` , which `update()` sets at the moment the screen turns green. Because `start_time` is used instead, every recorded value is `wait_delay + true reaction` . If the wait was 2400 ms and you reacted in 250 ms, the game records about 2650 ms. `go_time` is computed and stored but never used anywhere.

Defect 2: no state check, so false starts count as valid

`register_input()` never looks at `self.state` . Whatever the state is (including "waiting"), it computes a time, sets `self.state = "result"` , and returns it. So a click during the grey screen is recorded as a "reaction" (typically something small like 700 ms), the round ends, and the attempt counts toward the five rounds and the average. Nothing marks it as a false start.

The engine side contributes

The only guard in `GameEngine.handle_event` is:

python

```
if (is_click or is_space) and self.round.state != "result":
    reaction_ms = self.round.register_input()
```

```

self.reaction_times.append(reaction_ms)
self.result_shown_at = pygame.time.get_ticks()

```

It correctly blocks input during "result", but it allows "waiting" through. It also unconditionally appends whatever `register_input()` returns to `reaction_times`, so there is no way for a round to say "this input was invalid, don't record it."

Related to this, `rounds_total` is enforced by `len(self.reaction_times)` in `_start_next_round` and `round_num` in `render`. Once false starts are no longer recorded as reactions, a false-start round has to be handled explicitly (for example, replay that round without consuming one of the five), otherwise the round counter and game-over check will behave unexpectedly.

Summary of what needs to change for Task 1

Problem	Location	Needed fix
Time measured from <code>start_time</code>	<code>round.py</code> , <code>reaction_ms = now - self.start_time</code>	Measure from <code>self.go_time</code>
No state check on input	<code>round.py</code> , <code>register_input()</code>	Only produce a valid reaction when <code>state == "go"</code> ; flag input during "waiting" as a false start
Engine records every return value	<code>game_engine.py</code> , <code>handle_event</code>	Only append genuine reactions; handle false starts separately and keep round counting consistent

I haven't changed anything. When you're ready, I'd suggest I implement Task 1 first, with a distinct "false_start" state and its own message on screen, since Tasks 2 to 4 (results screen, replay, sound) all build on that state model.

2026-10-09 07:21:12 You 

Now implement Task 1 from the README: "Refine Input Timing Detection".

The README says reaction times are currently measured from when the round started, not from when the screen actually turned green, so every recorded time is inflated by the length of the wait phase. It also says clicking during the grey "wait" screen is timed and recorded exactly like a real reaction instead of being flagged as a false start.

Please fix this so that: 1. Reaction time is measured from the exact moment the screen turns green (the "go" moment), not from the round start. 2. Only genuine reactions after "go" are recorded in `reaction_times` and included in the average. 3. A click or Space press during the grey wait screen is flagged as a false start: no time is recorded, a "False start!" message is shown instead of a millisecond value, and that round is replayed after the short pause. 4. The next round still starts automatically after the short pause, and the session still ends after the configured number of rounds.change the code files and give



I'll write the two changed files (`main.py` needs no changes) and run a quick headless check.

I've changed `game/round.py` and `game/game_engine.py` . `main.py` is unchanged. I tested the logic headless with a short script and it behaved as described below. I haven't played it in a real window.

What changed

`game/round.py`

- There is a new `"false_start"` state alongside `"waiting"` , `"go"` and `"result"` .
- `register_input()` now checks the state first:
 - `"go"` : reaction time is `now - self.go_time` , measured from the moment the screen turned green. It sets `"result"` and returns the time.
 - `"waiting"` : it sets `"false_start"` and returns `None` , so nothing is timed.
 - **Any other state:** the input is ignored and it returns `None` .

`game/game_engine.py`

- `handle_event` only acts while the round is `"waiting"` or `"go"` . It appends to `reaction_times` only when `register_input()` returns a real time. For a false start it only starts the pause timer.
- `update` ends both `"result"` and `"false_start"` after the 800 ms pause, so the next round starts automatically.
- Replay and ending: a false start never adds to `reaction_times` . `_start_next_round` therefore just creates a fresh `Round` , and the round counter stays on the same number. The session still ends once `len(reaction_times) >= rounds_total` .
- `render` shows a red screen with "False start!" and a small "Too early - this round will be replayed" line instead of a millisecond value

Test results

- A click during the grey screen produced the false start, recorded nothing, and replayed the round.
- A reaction about 100 ms after "go" was recorded as 104 ms. It was not 200 ms plus that, as the old code would have recorded.
- The session ended after exactly 3 genuine reactions with `rounds_total=3` .

Notes

- `go_time` is set in `update()` , so it can be up to one frame (about 16 ms at 60 FPS) earlier than when green is actually drawn. That is normal for this loop structure.
- If a click and the end of the wait land in the same frame, the click counts as a false start. That is correct, because the player was still looking at grey.
- The console printout at game over is still there. Task 2 will replace it with a results screen.

2026-10-09 07:22:34

You



Task 1 works. Now implement Task 2 from the README:
"Implement Game Over Condition".

The README says: add a screen that displays the final results (every reaction time and the average) once all rounds are complete, then gracefully waits for input instead of just printing to the console.

Please make these changes: 1. When the configured number of rounds is complete, switch to a "Session Complete" screen drawn in the Pygame window (not printed to the console). 2. The screen lists every reaction time (for example "Round 1: 312 ms") and the final average. 3. The screen then waits for input. Pressing a key or clicking the mouse closes the game cleanly, and closing the window with the X also works. It must not freeze or exit by itself. 4. Don't change the Task 1 behavior: timing from the go moment, and false starts are not recorded and the round is replayed.

I'll make the changes in `game_engine.py` and `main.py`, then run a headless test and render the results screen to check the layout.

Task 2 is implemented in `game/game_engine.py` and `main.py`. `game/round.py` is unchanged. I ran it headless and checked the layout by rendering the screen to an image, but I haven't played it in a real window.

What changed

`game/game_engine.py`

- **Results screen:** a new `_render_results()` draws "Session Complete" in the Pygame window. It lists every time as "Round N: X ms" and shows "Average: X ms" in gold.
- **Console print removed:** the `print(...)` and `_game_over_logged` code are gone.
- **Layout:** up to 6 rounds use one column. More than 6 switch to two columns, so a longer Hard mode in Task 3 will still fit. I checked both 5 and 10 rounds.
- **Waiting for input:** once `game_over` is true, `handle_event` routes events to `_handle_results_event`. Any key press or mouse click (buttons 1 to 3) sets `engine.quit_requested = True`. The screen stays up until then and never exits by itself.
- **Input delay:** input is ignored for 600 ms after the results screen appears, so a leftover click or keypress from the last round can't dismiss it instantly. The "Press any key or click to exit" prompt only appears once input is accepted. Change `game_over_input_delay_ms` if you want a different delay.
- **Mouse wheel:** it is ignored, so scrolling won't close the game.

`main.py`

- The loop now stops when `engine.quit_requested` is set, then runs `pygame.quit()` as before. Closing the window with the X still works at any time, including on the results screen.

Checks

- **Task 1:** a false start was not recorded and replayed the round, and times were measured from the go moment.
- **Results screen:** after the last round it showed the results, did not exit by itself, and ignored input during the delay. A key or click then set `quit_requested`, and nothing was printed to the console.

Note for Task 3

Exiting via `quit_requested` is the simplest option for Task 2. For Task 3, the same results-screen hook is the natural place to show the difficulty menu (Easy, Medium, Hard) and a separate exit choice.

Task 2 works. Now implement Task 3 from the README: "Add Replay Option".

The README says: after the results screen, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard wait-time range/round count), or exit.

Please make these changes: 1. On the results screen, instead of just closing on a key press, show the options: press ENTER to play again, or ESC to exit. 2. Pressing ENTER opens a difficulty menu in the Pygame window: 23 / 40 1 = Easy, 2 = Medium, 3 = Hard. Each difficulty has its own wait-time range and number of rounds (for example Easy: longer, more predictable waits and fewer rounds; Hard: wider, less predictable waits and more rounds). Show these values on the menu. 3. Choosing a difficulty starts a fresh session: reset the reaction times, the average, the round counter and the false-start count. 4. Pressing ESC exits the game cleanly from the results screen and the menu, and closing the window with the X still works. 5. Don't break Task 1 (timing from the go moment, false starts replayed) or Task 2 (the results screen). easy shd be 3 rounds and 2.0-3.0 s wait and medium shd be 5 rounds and 1.0-3.0 s wait and hard shd be 10 rounds and 0.5-5.0 s wait



I'll implement the three difficulty presets and the menu and reset logic, then test the full replay flow headless and render the new screens.

The layout works for all three presets. With one column, a 6-round session would nearly touch the average line. No preset uses 6 rounds, but I'll switch to two columns above 5 rounds to be safe.

Task 3 is implemented in `game/game_engine.py` only. `main.py` and `game/round.py` are unchanged, since `main.py` already exits on `engine.quit_requested` and on the window X. I tested the whole flow headless and checked the new screens by rendering them to images, but I haven't played it in a real window.

I read "23 / 40 1 = Easy..." in your message as "1 = Easy, 2 = Medium, 3 = Hard".

What changed

Key	Level	Rounds	Wait range
1	Easy	3	2.0-3.0 s
2	Medium	5	1.0-3.0 s
3	Hard	10	0.5-5.0 s

Screens and flow:

- **Three screens:** the engine now tracks `mode` : "playing" , "results" or "menu" .
- **Results screen:** a key press or click no longer closes the game. ENTER opens the menu and ESC exits.
- **Difficulty menu:** it shows each preset's round count and wait range. Pressing 1, 2 or 3 (the numpad works too) starts a fresh session. ESC exits from here too.
- **Fresh session:** choosing a difficulty resets the reaction times (so the average goes back to 0), the round counter, the false-start count and the current round. It also applies that difficulty's round count and wait range.
- **False-start count:** this is new, because there was no counter before. It shows at the bottom-left while playing and on the results screen. The results screen also shows the difficulty name.
- **Starting difficulty:** the first game still starts on Medium, the same as before. Tell me if you'd rather show the menu at launch.

Unchanged: the Task 1 behaviour (timing from the go moment, false starts not recorded and replayed) and the Task 2 results screen. The 600 ms input delay on the results screen is kept, so leftover presses from the last round can't trigger anything.

Checks

- **Results screen:** input during the delay is ignored. Other keys and clicks do nothing, and ENTER opens the menu.
- **Menu:** a stray click, a letter key or ENTER does nothing.
- **Difficulty settings:** Easy, Medium and Hard each set the round count and wait range from your table. The first round's random wait stays inside the range, for example 500-5000 ms on Hard.
- **Fresh session:** after a session with a false start, choosing a difficulty reset the times, average and false-start count to zero.
- **Exit:** ESC quits from both the results screen and the menu.
- **Layout:** with 10 rounds on Hard, the two-column results screen fits cleanly. The 3- and 5-round layouts use a single column, and the menu screen looks right too.



Implement Task 4: Add Sound Feedback to the existing Pygame Reaction Time Tester with minimal changes.

- * Generate all three sounds programmatically in Python/Pygame without external audio files.
- * Play a short beep exactly when the screen turns green, without delaying the transition.
- * Play a short buzz when a click or SPACE press causes a false start during the grey waiting period. Preserve the existing behavior: record no reaction time and replay the round.
- * Play a short completion jingle once when the final round ends and the Session Complete/results screen appears.
- * Handle audio initialization failures or missing audio devices gracefully; the game must continue working silently without crashing.
- * Ensure sounds trigger only at their intended events, never repeatedly every frame, and do not interfere with reaction-time measurement.
- * Preserve all existing functionality from Tasks 1–3, including accurate reaction-time measurement, results and statistics, ENTER navigation, difficulty selection (1 = Easy, 2 = Medium, 3 = Hard), fresh sessions with correct settings, ESC exit, and window-close behavior.
- * Keep the existing code structure and file layout. Do not unnecessarily rewrite working functionality.

After implementation, briefly explain which files changed, where each sound is triggered, and how audio failures are handled.



Claude

I'll add the sound code to `game_engine.py` as a small helper class plus three one-line triggers, then test it with the real mixer, a mocked failing mixer, and an unavailable audio device.

Task 4 is done, and only `game/game_engine.py` changed. `main.py`, `game/round.py` and the file layout are untouched. I tested it headless, so I haven't heard the sounds on a real speaker.

Where each sound triggers

Sound	Trigger	Location in <code>game_engine.py</code>
Beep (880 Hz, 0.12 s)	The frame the round goes from grey to green	<code>update()</code> notes whether the round was "waiting" before <code>round.update()</code> . If it is now "go" , it plays the beep.
Buzz (140 Hz square wave, 0.25 s)	A click or Space press during the grey wait	<code>_handle_play_event()</code> , in the false-start branch right after <code>false_starts += 1</code>
Jingle (C-E-G-C ascending, 0.63 s)	When the final round ends and the results screen appears	<code>_start_next_round()</code> , right where <code>mode</code> becomes "results"

Each sound fires at its single event, not every frame. In the test the beep played once even after about 60 frames of waiting on green, and the jingle played once after about 100 frames on the results screen. `Sound.play()` is non-blocking. The beep is played after `go_time` is recorded, and genuine reactions never play a sound, so timing is unaffected.

How the sounds are made

A new `SoundBank` class, with a small `_synth` helper, builds the three sounds at startup from `math.sin` and square-wave samples. There are no audio files and no numpy. Generation takes about 30 ms. It adapts to the mixer's channel count and uses short fades so there are no clicks.

How audio failures are handled

- `SoundBank` sets up the mixer inside a `try/except` . If the mixer can't start, isn't available, or doesn't support 16-bit output, the sound list stays empty.
- `play()` does nothing for a missing sound and also catches errors from playback itself, so the game runs silently.
- I ran the full session flow three ways: with audio working, with `pygame.mixer.init` forced to raise an error, and with a nonexistent audio driver. All three behaved the same, with only the sounds missing in the last two.

Checks for Tasks 1 to 3

A false start still records nothing and replays the round. Reaction times, statistics, ENTER, difficulty selection and ESC all still work, and the Task 3 test passes unchanged.

All four tasks work now. Give me a short summary of what changed in each file for each task, and list any edge cases I should test before submitting



All four tasks are in place, and `main.py` was only touched in Task 2. I tested everything headless with simulated events and rendered screens, but I haven't played it in a real window or heard the sounds on a speaker.

What changed in each file

`game/round.py` (Task 1 only)

- **Task 1:** it adds the `"false_start"` state. `register_input()` now checks the state first. It times from `go_time` instead of `start_time`, flags input during `"waiting"` as a false start, and ignores input in any other state.

`game/game_engine.py` (all four tasks)

- **Task 1:**
 - Only genuine reactions are appended to `reaction_times`, and a false start is not recorded.
 - A false start shows a red "False start!" screen, and the round is replayed after the pause.
 - The session ends only after enough genuine reactions.
- **Task 2:**
 - It adds a "Session Complete" screen listing every round and the average.
 - It adds the 600 ms input delay.
 - The console `print` is removed.
- **Task 3:**
 - It adds the `mode` state (`playing`, `results`, `menu`).
 - It adds the `DIFFICULTIES` presets (Easy 3 rounds 2.0-3.0 s, Medium 5 rounds 1.0-3.0 s, Hard 10 rounds 0.5-5.0 s).
 - It adds the difficulty menu and the ENTER/ESC options on the results screen.
 - It adds a false-start counter and `_reset_session()` for fresh sessions.
- **Task 4:**
 - It adds `SoundBank`, which generates the sounds in code and fails silently without audio.
 - It adds three one-line triggers for the beep, buzz and jingle.

main.py (Task 2 only)

- The loop exits when `engine.quit_requested` is set, then runs `pygame.quit()` .

Edge cases to test before submitting

Timing and false starts

- Click or press Space during grey. You should see the red screen, no time recorded, and the same round number replayed.
- Make two or three false starts in a row. The false-start counter should climb and the round number should not advance.
- False start on the final round. The session should not end, and the round should replay.
- Mash clicks during the blue result pause. Nothing extra should be recorded.
- Sanity-check the times. Typical reactions should be about 200 to 400 ms, not 1000 ms or more, which would be the old bug.

Results screen and menu

- Click or press keys within the first half second of the results screen. Nothing should happen.
- Press other keys, click, or use the mouse wheel on the results screen and on the menu. Nothing should happen.
- ENTER opens the menu, and numpad Enter should work too. 1/2/3 and numpad 1/2/3 should each start the right difficulty.
- ESC should exit from both the results screen and the menu. Closing with the window X should work on the play, results and menu screens.

Replay

- Replay several times in a row, switching difficulty each time. The times, average, round counter and false-start count should reset every time, and the round count and wait range should match the preset.
- Check the 10-round Hard results screen (two columns) and the 3-round Easy one.

Sound

- You should hear one beep at green, one buzz per false start and one jingle when results appear, with no repeats.
- Check that a normal reaction plays no sound.
- Test with audio muted or unplugged. The game should still run silently.

Environment

- Alt-tab away and back mid-round. The click that refocuses the window may count as a false start.

One thing to check

The play-screen handler treats any `MOUSEBUTTONDOWN` as a click, and in pygame 2 the mouse wheel also generates those events. Scrolling during a round could therefore count as a false start or a reaction. I filtered the wheel out on the results screen but didn't touch the play behaviour. If you want it fixed, it's a one-line change to accept buttons 1 to 3 only in `_handle_play_event` .